

TorchDiff: A PyTorch Library for Denoising Diffusion Probabilistic Models and Beyond

Loghman Samani
samaniloqman91@gmail.com
Developer of TorchDiff

May 4, 2025

Abstract

Diffusion models are a powerful class of generative models that iteratively transform complex data distributions into simple noise distributions and learn to reverse this process to generate new samples. TorchDiff is an open-source Python library built on PyTorch, designed to implement popular diffusion models, including Denoising Diffusion Probabilistic Models (DDPM), Denoising Diffusion Implicit Models (DDIM), Score-Based Generative Models (SDE), and Latent Diffusion Models (LDM). This article introduces the theoretical foundations of diffusion models, focusing on DDPM, and demonstrates how to implement them using TorchDiff. We provide practical examples and invite contributions to the library's development.

1 Introduction to TorchDiff

TorchDiff is an open-source Python library built on the PyTorch API, designed to facilitate the implementation of state-of-the-art diffusion models. A C++ version is planned for future release to enhance performance. The library is grounded in seminal research, including:

- Denoising Diffusion Probabilistic Models (DDPM) by Ho et al. (2020),

- Denoising Diffusion Implicit Models (DDIM) by Song et al. (2021),
- Score-Based Generative Modeling through Stochastic Differential Equations (SDE) by Song et al. (2021), supporting Variance Exploding (VE), Variance Preserving (VP), and sub-Variance Preserving (sub-VP), and ODE methods.
- High-Resolution Image Synthesis with Latent Diffusion Models (LDM) by Rombach et al. (2022).

Version 1.0.0 of TorchDiff supports both conditional and unconditional training and sampling for these models. It includes a transformer-based text encoder, enabling conditioning on text prompts or class labels, making it suitable for applications like text-to-image generation. Readers are encouraged to visit the TorchDiff GitHub repository, explore the documentation at torchdiff.readthedocs.io, and contribute to its development.

2 Introduction to Diffusion Models

Diffusion models are a class of deep generative models that generate data by learning to reverse a process that gradually adds noise to a complex data distribution. Introduced by Sohl-Dickstein et al. (2015), the concept draws inspiration from non-equilibrium statistical physics, specifically the Jarzynski equality, which relates non-equilibrium processes to equilibrium free energy differences [1]. The core idea is to iteratively corrupt a data distribution (e.g., images) through a forward process, transforming it into a simple noise distribution, and then learn to reverse this process to generate new samples (Figure 1).

2.1 Historical Context

Historically, probabilistic generative models have faced a trade-off between tractability and flexibility. Tractable models, such as Gaussian distributions, are easy to analyze and fit but lack the expressiveness to capture complex patterns in high-dimensional data (e.g., images). Flexible models, such as those requiring Monte Carlo sampling, can model intricate distributions but are computationally expensive to train and evaluate [2]. Sohl-Dickstein et al. (2015) proposed diffusion probabilistic models to address this trade-off, offering both analytical tractability and the ability to model complex data distributions.

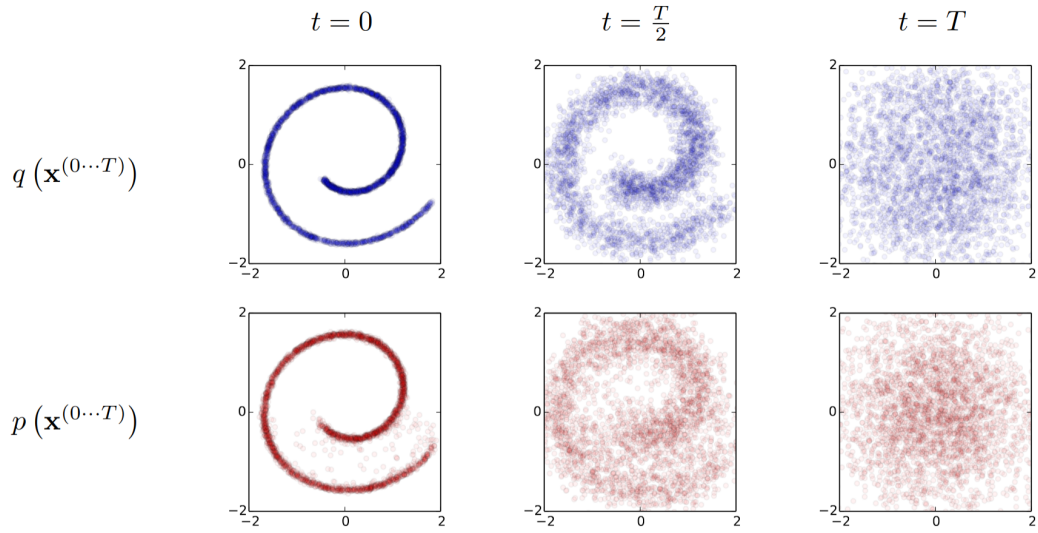


Figure 1: Forward and reverse processes of a diffusion model trained on 2D Swiss roll data. The top row shows the forward process $q(x^{(0...T)})$, where the data distribution (left) is gradually corrupted into an isotropic Gaussian (right). The bottom row shows the reverse process $p(x^{(0...T)})$, reconstructing the data distribution. Source: Sohl-Dickstein et al. (2015).

In their seminal work on DDPM, Ho et al. (2020) refined this approach by using a Markov chain to gradually transform a complex data distribution into a standard Gaussian distribution (Figure 2). By ensuring small noise increments at each step, the forward process becomes analytically tractable, enabling efficient training and high-quality sample generation.

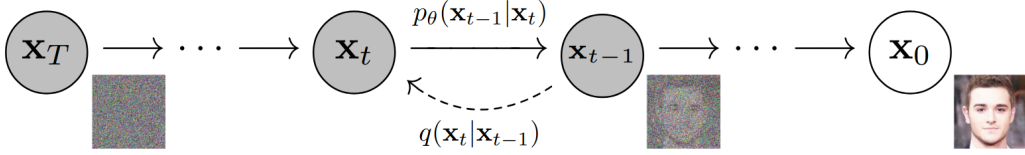


Figure 2: Forward and reverse processes of DDPM. The forward process (denoted by $q(x_t|x_{t-1})$, dashed arrows) gradually adds noise to the original image x_0 until it becomes pure noise x_T . The reverse process (denoted by $p_\theta(x_{t-1}|x_t)$, solid arrows) is learned to gradually denoise the sample from x_T back to x_0 , reconstructing the original image. Source: Ho et al. (2020).

3 Forward Process of Diffusion Models

The forward process in a diffusion model systematically transforms a complex data distribution, denoted $q(x^{(0)})$, into a simple, tractable distribution, typically an isotropic Gaussian $\pi(x^{(T)})$. This is achieved through a sequence of Markov diffusion kernels, each adding a small amount of Gaussian noise to the data. The process is inspired by non-equilibrium statistical physics, where structured information is gradually eroded, much like adding static to a radio signal until only noise remains [3].

Formally, the forward process is defined as a Markov chain over T steps, with the joint probability:

$$q(x^{(0:T)}) = q(x^{(0)}) \prod_{t=1}^T q(x^{(t)}|x^{(t-1)}) \quad (1)$$

Each transition $q(x^{(t)}|x^{(t-1)})$, known as a Markov diffusion kernel, is a Gaussian distribution that slightly attenuates the signal and adds noise:

$$q(x^{(t)}|x^{(t-1)}) = \mathcal{N}(x^{(t)}; \sqrt{1 - \beta_t}x^{(t-1)}, \beta_t I) \quad (2)$$

Here, $x^{(t-1)}$ is the data at step $t - 1$, $\sqrt{1 - \beta_t}$ scales the signal to preserve its magnitude, and $\beta_t I$ introduces Gaussian noise with variance β_t . The parameter β_t , known as the noise schedule, controls the amount of noise added at each step. Over T steps, this process transforms the data distribution into pure noise, approximating $\pi(x^{(T)}) \approx \mathcal{N}(0, I)$.

A key property of the forward process is its analytical tractability. For small β_t , the transitions are nearly deterministic, allowing the full chain to be evaluated efficiently. Additionally, the forward process can be expressed directly from the initial data $x^{(0)}$ to any step t using a reparameterization:

$$x^{(t)} = \sqrt{\bar{\alpha}_t}x^{(0)} + \sqrt{1 - \bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (3)$$

where $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$. This formulation, introduced by Ho et al. (2020), enables efficient sampling during training by directly computing $x^{(t)}$ without iterating through all intermediate steps [4].

3.1 Noise Schedule (β_t)

The noise schedule β_t is a critical hyperparameter that determines the rate at which noise is added. In the original diffusion model by Sohl-Dickstein et al. (2015), β_t was treated as a trainable parameter, optimized during training [3]. However, Ho et al. (2020) found that a fixed noise schedule, such as a linear or cosine schedule, simplifies training and improves sample quality [4]. A fixed schedule offers several advantages:

- **Stability:** A predefined schedule reduces the complexity of optimizing β_t , leading to more stable training dynamics.
- **Simplicity:** It eliminates the need for additional hyperparameters or learning algorithms for β_t .
- **Quality:** Carefully designed schedules, such as linear or cosine, ensure gradual noise addition, preserving structural information in early steps and achieving pure noise by step T .

In a linear schedule, β_t increases linearly from a small value (e.g., 10^{-4}) to a larger value (e.g., 0.02) over T steps. This gradual increase ensures that early steps preserve the data’s structure, allowing the model to learn meaningful features, while later steps produce near-pure noise. Table 1 illustrates a linear noise schedule for a DDPM model.

Table 1: Example linear noise schedule for DDPM with $T = 1000$, $\beta_1 = 10^{-4}$, and $\beta_T = 0.02$.

Step t	β_t	$\alpha_t = 1 - \beta_t$	$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$
1	10^{-4}	0.9999	0.9999
500	0.01	0.99	0.0821
1000	0.02	0.98	0.0002

3.2 Initializing the Forward Process in TorchDiff

The TorchDiff library, built on PyTorch, provides a modular interface for implementing diffusion models, including DDPM. To initialize the forward process for DDPM, users first define a set of hyperparameters using the `HyperParamsDDPM` class, which specifies the number of diffusion steps and the noise schedule. The following code snippet demonstrates this process:

```

1 import torch
2 from torchdiff.ddpm import HyperParamsDDPM
3
4 # Define DDPM hyperparameters
5 hp_ddpm = HyperParamsDDPM(
6     num_steps=1000,           # Number of diffusion steps
7     beta_start=1e-4,          # Initial noise scale
8     beta_end=0.02,            # Final noise scale
9     beta_method="linear"      # Linear noise schedule
10 )

```

Listing 1: Initializing DDPM hyperparameters in TorchDiff.

In Listing 1, `num_steps` determines the length of the forward process, during which the data is corrupted into Gaussian noise. The parameters `beta_start` and `beta_end` define the range of the noise schedule, starting with a small $\beta_1 = 10^{-4}$ to preserve data structure and increasing to $\beta_T = 0.02$ to ensure pure noise at step T . The `beta_method="linear"` argument specifies a linear interpolation for β_t ,

though TorchDiff also supports alternatives like cosine or sigmoid schedules for advanced use cases.

Next, the `ForwardDDPM` class is initialized using the defined hyperparameters to set up the forward diffusion process:

```
1 from torchdiff.ddpm import ForwardDDPM
2
3 # Initialize forward diffusion process
4 fwd_ddpm = ForwardDDPM(hp_ddpm)
```

Listing 2: Initializing the forward process in TorchDiff.

The `ForwardDDPM` class encapsulates the logic for applying the Gaussian diffusion kernels, computing intermediate states $x^{(t)}$, and managing the noise schedule. This modular design allows users to easily integrate the forward process into training and sampling pipelines, as demonstrated in subsequent sections.

4 Reverse Process of Diffusion Models

The reverse process in a diffusion model is the generative counterpart to the forward process, aiming to reconstruct the original data distribution $q(x^{(0)})$ from pure Gaussian noise $\pi(x^{(T)})$. Starting with a sample $x^{(T)} \sim \mathcal{N}(0, I)$, the model iteratively applies learned reverse transitions $p_\theta(x^{(t-1)}|x^{(t)})$ to denoise the data, gradually recovering the structure of the original distribution. This process is akin to restoring a blurred image by iteratively sharpening it, guided by a learned model.

Formally, the reverse process is defined as a Markov chain, with the joint probability:

$$p_\theta(x^{(0:T)}) = p(x^{(T)}) \prod_{t=1}^T p_\theta(x^{(t-1)}|x^{(t)}) \quad (4)$$

where $p(x^{(T)}) = \mathcal{N}(0, I)$ is the initial noise distribution, and each reverse transition is a Gaussian diffusion kernel parameterized by a neural network:

$$p_\theta(x^{(t-1)}|x^{(t)}) = \mathcal{N}(x^{(t-1)}; \mu_\theta(x^{(t)}, t), \Sigma_\theta(x^{(t)}, t)) \quad (5)$$

Here, $\mu_\theta(x^{(t)}, t)$ is the mean of the reverse transition, and $\Sigma_\theta(x^{(t)}, t)$ is the covariance, both predicted by a neural network parameterized by θ . In the original

diffusion model by Sohl-Dickstein et al. (2015), both the mean and covariance were learned [3]. However, in the DDPM framework by Ho et al. (2020), the covariance Σ_θ is fixed to a predefined schedule (e.g., $\Sigma_\theta = \beta_t I$), and the neural network focuses on predicting the mean μ_θ [4].

A key insight in DDPM is that instead of directly predicting the mean, the neural network, referred to as the *noise predictor*, is trained to estimate the noise component ϵ added during the forward process. This is mathematically equivalent because the mean μ_θ can be derived from the predicted noise using the forward process parameters. Specifically, for a sample $x^{(t)} = \sqrt{\bar{\alpha}_t}x^{(0)} + \sqrt{1 - \bar{\alpha}_t}\epsilon$, the noise predictor $\epsilon_\theta(x^{(t)}, t)$ estimates ϵ , and the mean is computed as:

$$\mu_\theta(x^{(t)}, t) = \frac{1}{\sqrt{\alpha_t}} \left(x^{(t)} - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x^{(t)}, t) \right) \quad (6)$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. This formulation simplifies training and leverages the forward process’s tractability, as discussed in Section 3.

4.1 Noise Predictor Architecture

In DDPM, the noise predictor is typically a U-Net-like neural network, which incorporates time embeddings and self-attention mechanisms to model the denoising process effectively [4]. The U-Net architecture consists of downsampling, bottleneck, and upsampling blocks, allowing it to capture both local and global features in the data (e.g., image textures and structures). Time embeddings, which encode the diffusion step t , are injected into the network to make the predictions time-conditional. Multi-head self-attention layers enhance the model’s ability to focus on relevant parts of the input, improving the quality of generated samples.

The noise predictor takes as input a noisy sample $x^{(t)}$ and the time step t , outputting the estimated noise $\epsilon_\theta(x^{(t)}, t)$. During training, the model learns to minimize the difference between the predicted noise and the actual noise added in the forward process, as detailed in Section ??.

4.2 Initializing the Reverse Process in TorchDiff

The TorchDiff library provides a straightforward interface for initializing the reverse process of DDPM. The `ReverseDDPM` class is used to set up the reverse diffusion process, leveraging the same hyperparameters defined for the forward process (Section 3.2). The following code snippet demonstrates this initialization:


```

1 from torchdiff.ddpm import ReverseDDPM
2
3 # Initialize reverse diffusion process
4 rev_ddpm = ReverseDDPM(hp_ddpm)

```

Listing 3: Initializing the reverse process in TorchDiff.

In Listing 3, the `ReverseDDPM` class uses the `hp_ddpm` object, which contains the noise schedule (β_t , α_t , $\bar{\alpha}_t$) and other parameters computed during forward process initialization. These parameters ensure consistency between the forward and reverse processes, enabling accurate denoising.

To implement the noise predictor, TorchDiff provides the `NoisePredictor` class, which mirrors the U-Net architecture described in Ho et al. (2020). The following code snippet initializes a noise predictor for RGB images:

```

1 from torchdiff.utils import NoisePredictor
2
3 # Initialize noise predictor model
4 noise_pred = NoisePredictor(
5     in_channels=3,                # Input channels (3 for RGB
6     down_channels=[32, 64, 128, 256], # Channels for
7     mid_channels=[256, 256, 256, 256], # Channels for
8     up_channels=[256, 128, 64, 32],   # Channels for
9     time_embed_dim=256,              # Dimension of time
10    y_embed_dim=256,                 # Dimension of conditional
11    num_down_blocks=2,               # Number of downsampling
12    num_mid_blocks=2,                # Number of bottleneck blocks
13    num_up_blocks=2                  # Number of upsampling blocks
14 )

```

Listing 4: Initializing the noise predictor in TorchDiff.

In Listing 4, the `NoisePredictor` is configured with a U-Net architecture tailored for RGB images (`in_channels=3`). The `down_channels`, `mid_channels`, and `up_channels` define the channel progression through the U-Net’s encoder, bottleneck, and decoder, respectively. The `time_embed_dim` parameter specifies the size of the time embeddings, enabling the model to condition predictions on

the diffusion step t . The `y_embed_dim` parameter supports conditional generation (e.g., text prompts), which is discussed further in Section 5. The number of blocks (`num_down_blocks`, `num_mid_blocks`, `num_up_blocks`) controls the depth of the network, balancing computational complexity and model capacity.

5 Training Objective

The training objective of a diffusion model is to optimize the model parameters θ to maximize the likelihood of the observed data under the generative model. This involves computing the probability $p_\theta(x^{(0)})$ that the model assigns to real data points $x^{(0)}$, measuring how well the model captures the true data distribution. This section outlines the general approach to maximizing log-likelihood, the KL divergence-based loss formulation from Sohl-Dickstein et al. (2015), and the simplified loss introduced by Ho et al. (2020) for DDPM [3, 4].

5.1 Maximizing Log-Likelihood

A key question in training diffusion models is how to evaluate the model’s fit to the data. The goal is to maximize the expected log-likelihood over the true data distribution $q(x^{(0)})$:

$$\mathcal{L} = \mathbb{E}_{q(x^{(0)})} [\log p_\theta(x^{(0)})] \quad (7)$$

The generative model defines the probability $p_\theta(x^{(0)})$ through the joint distribution over the full trajectory:

$$p_\theta(x^{(0)}) = \int dx^{(1:T)} p_\theta(x^{(0:T)}) = \int dx^{(1:T)} p(x^{(T)}) \prod_{t=1}^T p_\theta(x^{(t-1)} | x^{(t)}) \quad (8)$$

However, computing this integral over all possible noise trajectories is intractable due to the high-dimensional nature of $x^{(1:T)}$. To address this, diffusion models leverage the forward process $q(x^{(1:T)} | x^{(0)})$, which is analytically tractable, as discussed in Section 3. Using importance sampling, the likelihood can be rewritten as:

$$p_\theta(x^{(0)}) = \int dx^{(1:T)} q(x^{(1:T)}|x^{(0)}) \frac{p_\theta(x^{(0:T)})}{q(x^{(1:T)}|x^{(0)})} \quad (9)$$

This allows estimation of $p_\theta(x^{(0)})$ by sampling trajectories from the forward process and computing the ratio of reverse and forward transition probabilities:

$$p_\theta(x^{(0)}) \approx p(x^{(T)}) \prod_{t=1}^T \frac{p_\theta(x^{(t-1)}|x^{(t)})}{q(x^{(t)}|x^{(t-1)})} \quad (10)$$

When the noise steps β_t are small, the forward and reverse processes are nearly symmetric, and this approximation becomes accurate with a single trajectory sample, resembling a quasi-static process inspired by the Jarzynski equality [1].

To make training tractable, the log-likelihood is bounded using Jensen’s inequality, yielding the evidence lower bound (ELBO):

$$\log p_\theta(x^{(0)}) \geq \mathbb{E}_{q(x^{(1:T)}|x^{(0)})} \left[\log \frac{p_\theta(x^{(0:T)})}{q(x^{(1:T)}|x^{(0)})} \right] \quad (11)$$

Taking the expectation over $q(x^{(0)})$, the training objective becomes maximizing the ELBO:

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q(x^{(0:T)})} \left[\log \frac{p_\theta(x^{(0:T)})}{q(x^{(1:T)}|x^{(0)})} \right] \quad (12)$$

This ELBO measures how closely the learned reverse process p_θ matches the true posterior q , providing a practical objective for optimization.

5.2 KL Formulation of the Loss (2015)

Sohl-Dickstein et al. (2015) decomposed the ELBO into a sum of Kullback-Leibler (KL) divergences and entropy terms, offering a theoretically grounded loss function [3]. The KL divergence measures the difference between two probability distributions, here comparing the true posterior $q(x^{(t-1)}|x^{(t)}, x^{(0)})$ to the learned reverse transition $p_\theta(x^{(t-1)}|x^{(t)})$. The ELBO can be expressed as:

$$\begin{aligned}\mathcal{L}_{\text{ELBO}} = & - \sum_{t=2}^T \mathbb{E}_{q(x^{(0)}, x^{(t)})} \left[D_{\text{KL}} \left(q(x^{(t-1)} | x^{(t)}, x^{(0)}) \parallel p_{\theta}(x^{(t-1)} | x^{(t)}) \right) \right] \\ & + H_q(x^{(T)} | x^{(0)}) - H_q(x^{(1)} | x^{(0)}) - H_p(x^{(T)})\end{aligned}\quad (13)$$

where:

- $D_{\text{KL}}(q \parallel p)$: The KL divergence between the true and learned reverse transitions at each step t .
- $H_q(x^{(t)} | x^{(0)})$: Conditional entropy terms capturing uncertainty in the forward process.
- $H_p(x^{(T)})$: Entropy of the initial noise distribution, typically a constant for $p(x^{(T)}) = \mathcal{N}(0, I)$.

Training involves minimizing the sum of KL divergences, effectively aligning the reverse process with the true data-generating process. This formulation is computationally intensive due to the need to estimate multiple KL terms, but it provides a robust framework for learning flexible diffusion models.

5.3 Simplified DDPM Loss (2020)

Ho et al. (2020) introduced a simplified loss function for DDPM that significantly reduces computational complexity while maintaining high sample quality [4]. They proposed three key simplifications:

- **Fixed Noise Schedule:** The forward process noise schedule β_t is predefined (e.g., linear or cosine), as discussed in Section 3.1.
- **Fixed Reverse Variance:** The reverse process covariance $\Sigma_{\theta}(x^{(t)}, t)$ is set to a constant (e.g., $\beta_t I$) rather than learned.
- **Noise Prediction:** The neural network predicts the noise ϵ added in the forward process instead of the reverse mean μ_{θ} .

Recall from Section 3 that the forward process generates a noisy sample:

$$x^{(t)} = \sqrt{\bar{\alpha}_t} x^{(0)} + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (14)$$

where $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$. The noise predictor $\epsilon_\theta(x^{(t)}, t)$ is trained to estimate ϵ given $x^{(t)}$ and t . The simplified loss is the mean squared error (MSE) between the true and predicted noise:

$$L_{\text{simple}}(\theta) = \mathbb{E}_{t, x^{(0)}, \epsilon} \left[\left\| \epsilon - \epsilon_\theta \left(\sqrt{\bar{\alpha}_t} x^{(0)} + \sqrt{1 - \bar{\alpha}_t} \epsilon, t \right) \right\|_2^2 \right] \quad (15)$$

This loss approximates the ELBO because predicting the noise ϵ is equivalent to predicting the reverse mean μ_θ , as shown in Section 4. The simplification reduces the training objective to a straightforward regression task, making it computationally efficient and empirically effective. Ho et al. (2020) demonstrated that this loss produces high-quality samples, comparable to or better than the full KL-based loss, due to its focus on denoising performance.

Table 2 summarizes the differences between the 2015 and 2020 loss formulations.

Table 2: Comparison of loss formulations for diffusion models.

Aspect	Sohl-Dickstein et al. (2015)	Ho et al. (2020)
Objective	Sum of KL divergences (ELBO)	Simplified MSE on noise prediction
Noise Schedule	Trainable β_t	Fixed (e.g., linear, cosine)
Reverse Variance	Learned Σ_θ	Fixed (e.g., $\beta_t I$)
Prediction Target	Mean and variance of reverse transition	Noise component ϵ
Computational Complexity	High (multiple KL terms)	Low (single MSE term)

5.4 Training with TorchDiff

With the theoretical foundations of DDPM established, this section demonstrates how to train a conditional DDPM model using the TorchDiff library. Training involves preparing a dataset, configuring a conditional text encoder, defining evaluation metrics, and setting up the training pipeline. The following steps outline the process, focusing on a practical example with ImageNet data.

First, the dataset is prepared using PyTorch’s `torchvision` utilities to load and preprocess images. The images are normalized to the range $[-1, 1]$, which is standard for diffusion models to ensure stable training:

```

1 from torchvision import datasets, transforms
2 from torch.utils.data import DataLoader
3
4 # Define image preprocessing transformations
5 transform = transforms.Compose([
6     transforms.Resize(224),          # Resize to 224x224
7     transforms.CenterCrop(224),      # Crop to center
8     transforms.ToTensor(),           # Convert to tensor
9     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)) #
10     # Normalize to [-1, 1]
11 ])
12
13 # Load ImageNet training and validation datasets
14 train_dataset = datasets.ImageFolder(root='./imagenet/train',
15                                     transform=transform)
16 val_dataset = datasets.ImageFolder(root='./imagenet/val',
17                                   transform=transform)
18
19 # Create data loaders
20 train_loader = DataLoader(
21     train_dataset, batch_size=64, shuffle=True, num_workers
22     =8, pin_memory=True
23 )
24 val_loader = DataLoader(
25     val_dataset, batch_size=64, shuffle=False, num_workers=8,
26     pin_memory=True
27 )

```

Listing 5: Preparing the ImageNet dataset for training.

In Listing 5, the transform pipeline resizes and normalizes images, while the `DataLoader` objects facilitate efficient batch processing. The `pin_memory=True` option optimizes data transfer to GPUs, enhancing training performance.

Next, a conditional text encoder is initialized to enable text-prompt-based generation, such as generating images from descriptions like “a cat” or “a dog.” TorchDiff’s `TextEncoder` class leverages a pre-trained BERT model to encode text prompts into embeddings:

```

1 from torchdiff.utils import TextEncoder
2
3 # Initialize text encoder
4 text_enc = TextEncoder(
5     use_pretrained_model=True,          # Use pre-trained BERT
6     model_name="bert-base-uncased",    # BERT model variant
7     output_dimension=256                # Embedding dimension
8 )

```

Listing 6: Initializing the text encoder for conditional generation.

In Listing 6, the `TextEncoder` produces 256-dimensional embeddings, which are used to condition the noise predictor during training and sampling. This enables the model to generate images that align with specific text prompts, enhancing its flexibility for applications like text-to-image synthesis.

To evaluate the model’s performance during training, TorchDiff provides the `Metrics` class, which supports several image quality metrics, including Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index (SSIM), Fréchet Inception Distance (FID), and Learned Perceptual Image Patch Similarity (LPIPS). These metrics compare generated images to ground-truth images, providing quantitative insights into model performance:

```

1 from torchdiff.utils import Metrics
2
3 # Initialize metrics for evaluation
4 metrics = Metrics(
5     device="cuda",          # Use GPU for computations
6     fid=True,               # Compute FID score
7     ssim=True,              # Compute MSE, PSNR, and SSIM
8     lpips_=True             # Compute LPIPS using VGG backbone
9 )

```

Listing 7: Configuring evaluation metrics.

In Listing 7, `fid=True` enables FID computation, which measures the similarity between generated and real image distributions using a pre-trained Inception V3 model. Similarly, `ssim=True` and `lpips_=True` activate structural and perceptual similarity metrics, respectively.

Finally, the `TrainDDPM` class orchestrates the training process, combining the noise predictor, text encoder, hyperparameters, and evaluation metrics. The training objective is the simplified MSE loss discussed in Section 5.3, optimized using the Adam optimizer:

```

1 import torch.nn as nn
2 from torchdiff.ddpm import TrainDDPM
3
4 # Configure optimizer
5 optim = torch.optim.Adam(
6     list(noise_pred.parameters()) + list(text_enc.parameters
7         ()), # Optimize both models
8     lr=1e-4 # Learning rate
9 )
10 # Define MSE loss function
11 loss_fn = nn.MSELoss()
12
13 # Initialize training pipeline
14 trainer = TrainDDPM(
15     noise_predictor=noise_pred, # Noise predictor model
16     hyper_params=hp_ddpm, # Hyperparameters
17     conditional_model=text_enc, # Text encoder for
18         conditioning
19     metrics_=metrics, # Evaluation metrics
20     optimizer=optim, # Adam optimizer
21     objective=loss_fn, # MSE loss function
22     data_loader=train_loader, # Training data loader
23     val_loader=val_loader, # Validation data loader
24     max_epoch=100, # Number of training
25         epochs
26     device="cuda", # Use GPU
27     store_path="ddpm_model.pth", # Path to save model
28         checkpoints
29     val_frequency=5 # Evaluate every 5 epochs
30 )
31
32 # Start training
33 trainer()

```

Listing 8: Setting up and starting DDPM training.

In Listing 8, the TrainDDPM class integrates all components, training the noise predictor and text encoder to minimize the MSE between predicted and actual noise. The `val_frequency=5` parameter triggers validation every five epochs, using the metrics defined earlier to monitor performance. The trained model is saved to `ddpm_model.pth` for later use in sampling.

5.5 Sampling with TorchDiff

Once trained, the DDPM model can generate new samples, either unconditionally or conditioned on text prompts, using the reverse process described in Section 4. TorchDiff’s `SampleDDPM` class simplifies this process, allowing users to generate images by specifying the model, text prompts, and output settings. The following code snippet demonstrates generating three conditional images:

```
1 from torchdiff.ddpm import SampleDDPM
2
3 # Initialize sampler
4 sampler = SampleDDPM(
5     reverse_diffusion=rev_ddpm,      # Reverse process
6     noise_predictor=noise_pred,      # Noise predictor model
7     image_shape=(224, 224),          # Output image dimensions
8     conditional_model=text_enc,      # Text encoder for
9         conditioning                  # conditioning
10    tokenizer="bert-base-uncased",    # Tokenizer for text
11        prompts                       # prompts
12    batch_size=3,                     # Generate three images
13    in_channels=3,                    # RGB images
14    device="cuda",                    # Use GPU
15    output_range=(-1, 1)              # Output range matching
16        training data                 # training data
17)
18
19 # Generate images from text prompts
20 images = sampler(
21     conditions=["a cat", "a dog", "a box"], # Text prompts
22     save_images=True,                       # Save generated
23         images                           # images
24     save_path="ddpm_generated"             # Output
25         directory                        # directory
26)
```

Listing 9: Generating conditional images with TorchDiff.

In Listing 9, the `SampleDDPM` class uses the trained noise predictor and text encoder to generate 224x224 RGB images conditioned on the prompts “a cat,” “a dog,” and “a box.” The `output_range=(-1, 1)` ensures the generated images match the training data’s normalization. The `save_images=True` option saves the images to the `ddpm_generated` directory, facilitating visual inspection.

Table 3 summarizes the key TorchDiff classes used in training and sampling, highlighting their roles and configurations.

Table 3: Key TorchDiff classes for DDPM training and sampling.

Class	Role	Key Parameters
HyperParamsDDPM	Defines diffusion hyperparameters	num_steps, beta_start, beta_end, beta_method
ForwardDDPM	Implements forward process	hyper_params
ReverseDDPM	Implements reverse process	hyper_params
NoisePredictor	Predicts noise for denoising	in_channels, down_channels, time_embed_dim
TextEncoder	Encodes text prompts	model_name, output_dimension
Metrics	Evaluates generated images	fid, ssim, lpips_
TrainDDPM	Orchestrates training	noise_predictor, conditional_model, max_epoch
SampleDDPM	Generates samples	reverse_diffusion, image_shape, conditions

6 Conclusion

Diffusion models, exemplified by Denoising Diffusion Probabilistic Models (DDPM), offer a powerful framework for generative modeling, balancing tractability and flexibility to produce high-quality samples. This article has provided a comprehensive introduction to diffusion models, covering their theoretical foundations—forward and reverse processes, training objectives—and practical implementation using the TorchDiff library. TorchDiff simplifies the development of DDPM and other diffusion models (e.g., DDIM, SDE, LDM) by providing modular, PyTorch-based classes for configuring, training, and sampling.

Key contributions of TorchDiff include its support for both conditional and unconditional generation, a flexible text encoder for text-to-image tasks, and a suite of evaluation metrics to monitor performance. The library’s design, grounded in seminal research [3, 4, 5, 7], makes it accessible to researchers and practitioners alike. Future developments, including a planned C++ implementation, promise to enhance performance and broaden its applicability.

Readers are encouraged to explore TorchDiff at github.com/LoqmanSamani/TorchDiff, contribute to its development (e.g., by adding new models or optimizing code), and leverage its capabilities for generative modeling tasks. By bridging theory and practice, TorchDiff empowers the community to advance the field of diffusion models.

References

- [1] Jarzynski, C. (1997). Nonequilibrium equality for free energy differences. *Physical Review Letters*, 78(14), 2690.
- [2] Neal, R. M. (2001). Annealed importance sampling. *Statistics and Computing*, 11(2), 125–139.
- [3] Sohl-Dickstein, J., Weiss, E. A., Maheswaranathan, N., & Ganguli, S. (2015). Deep unsupervised learning using nonequilibrium thermodynamics. *International Conference on Machine Learning (ICML)*, 2256–2265.
- [4] Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 6840–6851.
- [5] Song, J., Meng, C., & Ermon, S. (2021). Denoising diffusion implicit models. *International Conference on Learning Representations (ICLR)*.
- [6] Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., & Poole, B. (2021). Score-based generative modeling through stochastic differential equations. *International Conference on Learning Representations (ICLR)*.
- [7] Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 10684–10695.